

# Meta에서 배운 실전 테크닉 - AI가 길 잃지 않는 코드베이스 & 토큰 비용 최적화 [Wlt7GhA\_14s]

## Meta에서 배운 실전 테크닉: AI가 길 잃지 않는 코드베이스와 토큰 비용 최적화

### 개요

클로드 코드(Claude Code) 에이전틱 엔지니어링 강의의 일부로, 실리콘밸리 메타(Meta) 현장에서 쓰이는 두 가지 실전 테크닉을 다룬다. 첫째는 AI 에이전트가 길을 잃지 않는 “AI 레디(AI-ready) 코드베이스”를 설계하는 법과 이를 점수화하는 스킬이고, 둘째는 에이전트를 쓸수록 늘어나는 토큰 비용을 측정하고 통제하는 법(토큰 대시보드와 코스트 게이트)이다. 핵심은 이런 개선을 개인이 아니라 팀 공유 자산으로 만들어 조직 전체의 효율을 올리는 것이다.

### 1. AI 레디 코드베이스란 무엇인가

- CLAUDE.md가 에이전트의 “헌법(행동 지침)”이라면, 코드베이스는 에이전트가 실제로 일하는 “지형”이다. 지침이 아무리 좋아도 지형이 미로면 에이전트는 엉뚱한 파일을 읽고, 같은 함수를 두 번 만들고, 기존 유틸을 못 찾고 새로 만드는 식으로 길을 잃는다.
- AI 레디 코드베이스 = 에이전트가 길을 잃지 않고 작업할 수 있는 코드베이스. “좋은 코드베이스”와는 살짝 다르다.
- 사람은 한 달쯤 일하면 머릿속에 코드베이스 지도가 그려지지만, 에이전트는 매 세션마다 처음 들어오는 신규 입사자라고 봐야 한다. 사람이 익숙함으로 메우던 탐색 비용을 에이전트는 매번 새로 치른다. 그래서 기준이 더 빡빡하다. 신규 입사자가 일주일이면 적응하는 수준으로는 부족하고, 첫 1분 안에 어디로 가야 할지가 명확해야 한다.

#### 핵심 통찰: 실패의 대부분은 코드 품질이 아니라 탐색·이해 비용

- 현장에서 에이전트 실패의 대부분은 코드를 잘못 짜서가 아니라, 올바른 위치를 못 찾고, 기존 유틸을 못 보고, 진짜 진입점을 몰라서 발생한다.
- 이것은 모델이 더 똑똑해진다고 해결되는 문제가 아니라, 코드베이스 지형 자체가 정리돼야 하는 문제다.
- 예시: “결제 로직을 수정해라”라는 태스크를 줬을 때, AI 레디가 안 된 리포지토리에서는 payment 관련 47개 파일이 잡히는데 진입점을 몰라 헤매고, 중복(duplicate) 코드 중 뭐가 진짜인지 몰라 엉뚱한 걸 고치고, 테스트를 돌리면 다른 게 깨져 다시 탐색한다. 결국 토큰의 약 80%가 탐색에 쓰이고, 실제 코드 수정은 약 20%뿐이다. AI 레디 코드베이스는 이 80%를 다시 코드 수정 쪽으로 돌려놓는 작업이다.

### 2. AI 레디로 만드는 두 단계: 세니티(건강성) → 카토그래피(지도)

순서가 중요하다. 건강하지 않은 코드 위에 지도를 그리면 지도가 거짓말을 한다. 둘 다 필요하다.

| 단계 | 이름                               | 하는 일                                    | 왜 필요한가                                     |
|----|----------------------------------|-----------------------------------------|--------------------------------------------|
| 1  | 코드베이스 세니티<br>(건강성)               | 테스트 커버리지, 데드코드 제거,<br>컨벤션 일관성, 코드 스멜 제거 | 에이전트가 잘못된 코드를 따라 들어가 시간<br>낭비하지 않도록 선결로 확보 |
| 2  | 카토그래피<br>(cartography, 지도<br>제작) | 건강한 지형 위에 지도를 얹어<br>탐색 비용을 한 번 더 낮춤     | 에이전트의 탐색 비용을 추가로 절감하는 가속<br>장치             |

#### 에이전트 시대에 테스트·컨벤션·데드코드가 더 중요해진 이유

테스트/컨벤션/데드코드 제거 자체는 새로운 얘기가 아니지만, 에이전트 시대에는 가치의 의미가 살짝 달라진다.

| 항목          | 사람 시대의 가치              | 에이전트 시대의 가치                                 |
|-------------|------------------------|---------------------------------------------|
| 테스트<br>커버리지 | 회귀(regression) 방지, 안전망 | 에이전트가 <b>본인이 잘못된 걸 스스로 알게 하는 셀프 검증 신호</b>   |
| 컨벤션 일관성     | 가독성                    | 에이전트가 같은 패턴을 일반화해 재사용 가능(여러 컨벤션이면 헛갈려 잘못 씀) |
| 데드코드 제거     | 유지보수, 버그 감소            | 에이전트가 <b>잘못된 길로 갈 수 없게</b> 차단               |
| 코드 스멜 제거    | 코드 품질                  | 에이전트가 나쁜 코드를 따라 쓰지 않게 함                     |

- 에이전트는 기본적으로 루프를 돌며 본인이 잘못된 걸 셀프 코렉트(self-correct)하는 능력이 있는데, 그 능력이 발휘되려면 **테스트가 잘 커버된 코드베이스**여야 훨씬 좋은 퍼포먼스를 낸다.

### 카토그래피 vs 리드미(README)의 차이

지도는 리드미와 대상부터 다르다.

| 구분 | README | 카토그래피(지도)                                                                                    |
|----|--------|----------------------------------------------------------------------------------------------|
| 대상 | 사람     | 에이전트                                                                                         |
| 내용 | 서술형 문서 | 진입점, 의존성 그래프(dependency graph), 도메인 용어집, 가처(gotcha, 잘못된/함정 패턴), 뽀뽀하고 정확한 정보, 코드 변경마다 갱신되는 지도 |

## 3. 스킬 실습: AI Readiness Cartography (코드베이스 점수화)

- 강사가 만든 **AI Readiness Cartography 스킬로**, 코드베이스가 얼마나 AI 레디한지를 먼저 점수화한다(CLAUDE.md 개선 때와 마찬가지로 “먼저 점수를 매긴다”는 접근).
- 이건 린트(lint)가 아니라, **에이전트가 어느 카테고리에서 가장 길을 잃는지**를 보여주는 리포트다.
- 100점 만점, **7개 카테고리**로 감사(audit)한다. 언급된 카테고리 예: 내비게이션(탐색), 컨텍스트 품질, 암묵 지식 문서화, 의존성 매핑, 검증(verification), 프레시니스(freshness, 최신성), 에이전트 아웃컴.
- 워크플로우: 파이썬 스크립트가 채점 → HTML 대시보드 산출 → **무엇을 바꿔야 할지(개선 액션)**까지 알려줌.
- 루브릭은 절대적인 게 아니라 강사가 임의로 만든 것이므로, **본인/팀 코드베이스에 맞게 카테고리를 넣거나 빼서 스크립트를 다시 작성**하면 된다. “채점지는 팀이 만드는 것.”

채점지(루브릭)를 이 파이썬 스크립트에 맞춰 다시 작성해 줘

### 데모 결과 해석

- 데모 프로젝트(카카오톡 채팅 분석 웹 리포)를 감사한 결과 **AI 준비도 54점** → “**에이전트에게 취약**”.
- 탐색 커버리지는 나쁘고, 컨텍스트 문서 품질은 꽤 괜찮은 편, 나머지는 대체로 낮음.
- 구조 지도는 CLAUDE.md 2개와 ARCHITECTURE.md 등 문서 및 파일 연결 관계를 시각화해 보여줌.
- 제시된 **최우선 ROI 액션(개선 제안)** 예:
  - 라이브러리/컴포넌트/모듈 레벨의 CLAUDE.md를 만들어라 (CLAUDE.md를 더 잘게 쪼개라)
  - 린트·타입체크·테스트·validate를 PR에서 자동 실행되게 만들어라
  - Mermaid 흐름도를 ARCHITECTURE.md에 넣어 시각화해라 (시간 절감)
  - ADR(Architecture Decision Record)을 도입해 암묵 지식을 외부화해라
  - 코드 오너(CODEOWNERS)·PR 검증 체크리스트 템플릿을 만들어라

### 반드시 지킬 원칙: 주기화 + PR 생성까지

- 한 번 돌리는 건 효과가 별로 없다. **크론잡(cron job)으로 주기적으로(예: 2주~한 달에 한 번)** 돌리고, ROI가 있으면 PR을 자동 생성하게 해야 한다.
- 리포트 자동 생성만으로는 의미가 없고, **고쳐야(반영해야)** 의미가 있다. PR을 만들어 실제로 머지(land)하는 것까지가 완성.

## 4. 토큰과 비용의 기초

자산(플러그인, 세컨드 브레인, 스펙 등)이 쌓이면 컨텍스트가 늘고, 컨텍스트가 늘면 토큰이 늘고, 결국 청구서(비용)가 커진다. CLAUDE.md에 수천 줄을 박아두면 팀원 한 명 한 명이 세션을 열 때마다 그 전체가 컨텍스트에 들어가, 한 세션에 수십만 토큰이 우습게 나온다.

### 토큰이란

- 토큰은 모델이 텍스트를 처리하는 최소 단위. 영어 단어 하나가 토큰 하나가 아니다.
- BPE(Byte Pair Encoding)** 사전에 맞춰 문자열이 토큰 시퀀스로 잘린다. 예: `tokenizer` → `token` + `izer` (2개), `subwords` → 3개.
- 대략적인 감(정확치는 아님):
  - 영어: 1토큰 ≈ 4글자 ≈ 0.75단어
  - 한국어: 1토큰 ≈ 1.5~2글자 (영어보다 약 1~2배 더 소모)
  - 코드: 영어와 비슷한 수준
- 따라서 토큰을 완전 최적화하고 싶으면 **영어로 쓰는 게 유리**하다는 점을 비용 차원의 의사결정으로 알아두면 좋다(한국어로 써도 무방).

### 대략적 가격 감각 (강의 시점 기준, 정확한 수치는 Anthropic 페이지 확인)

| 항목                  | 대략 단가 (per 1B tokens 기준으로 언급됨)    |
|---------------------|-----------------------------------|
| 입력(input) 토큰        | 약 3불                              |
| 출력(output) 토큰       | 약 15불 (출력이 더 비쌈)                  |
| 캐시 라이트(cache write) | 약 3.75불 (새 정보를 캐시에 넣을 때, 조금 더 비쌈) |
| 캐시 리드(cache read)   | 약 0.3불 (거의 1/10, 훨씬 싼)            |

주: 강사가 구두로 제시한 숫자(오/육/이오 불 등)는 발음이 뭉개져 정확 매핑이 어렵다. 정확한 최신 단가는 반드시 Anthropic 공식 페이지에서 확인할 것. 요지는 “출력 > 입력 > 캐시 라이트 ≫ 캐시 리드” 순으로 비싸고, 캐시 리드가 압도적으로 싸다는 것.

- 규모 예시: 한 사람이 하루 입력 500K / 출력 100K ≈ 5불 → 한 달 20영업일 ≈ 100불 → 팀 규모면 월 1,000불 → 연 12,000불. 조직 (메타는 수만 명 규모)이 되면 **기하급수적**으로 커진다. → **캐싱을 잘 써야 한다**.

### 누구의 청구서를 줄이는가

- 개인 청구서**: 내 카드로 결제하니 비용이 눈에 보여 자연스럽게 보수적으로 씬.
- 팀/회사 청구서**: 비용이 안 느껴져 마음껏 쓰게 됨. 하지만 리더십에서 토큰 최적화 가이드라인이 점점 내려온다. 결국 조직 차원의 최적화가 필요.

## 5. 프롬프트 캐시(prompt cache): 비용을 실제로 깎는 최대 도구

- 정의**: 같은 프리픽스(prefix)를 반복해서 보내면 Anthropic 서버가 그 계산 결과를 캐시해 둔다. 다음 호출에서 같은 프리픽스가 필요하면 입력 토큰이 아니라 **캐시 리드로 약 1/10 가격**에 읽는다.
- 합의**: CLAUDE.md, 시스템 프롬프트처럼 매 턴 거의 똑같이 들어가는 것들은 무조건 캐시를 잘 활용해야 한다. 안 쓰면 매번 풀 가격으로 계산된다.

### 캐시 구조 3가지 핵심

- TTL(Time To Live) ≈ 5분**: 5분 후 캐시가 리셋된다(정확 값은 Anthropic 확인). 저장은 잠깐만 유지됨.
- 마지막 히트에서 5분 연장**: 계속 작업하면 괜찮지만, 밥/커피로 5분 넘게 자리를 비우면 캐시가 죽는다. → 꼭 이어서 하는 게 좋고, 롱러닝(long-running) SDD(스펙 주도 개발) 같은 방식이 실제로 토큰 효율이 좋다.
- 프리픽스 기반**: 앞쪽의 시스템 프롬프트·CLAUDE.md는 캐시되고, 매번 새로 들어가는 유저 메시지(“이거 고쳐, 저거 수정해”)는 매번 새로 라이트된다.

## 절대 하지 말 것: 세션 중간에 프리픽스 바꾸기

- 캐시는 프리픽스가 바뀌면 전부 날아간다(예: 저장된 값 A가 A-1로 바뀌면 이전 캐시를 재사용 못 함).
- 세션 중간에 CLAUDE.md를 고치지 마라. 항상 작업이 다 끝난 뒤 고쳐라. 중간에 고치면 토큰 효율이 급격히 나빠진다.
- 세션 중간에 모델을 바꾸지 마라. 모델도 시스템 프롬프트에 들어가므로, Opus 쓰다 중간에 Sonnet으로 바꾸면 앞의 캐시가 전부 날아가 처음부터 다시 입력 토큰으로 들어간다.
- 세션을 재개(resume)할 때 클로드가 “이 세션 토큰이 너무 큰데 확실하냐? 처음부터 다시 할래, 콤팩트(compact)할래?”라고 묻는 이유가 이 캐시 때문이다. 작업 중엔 캐시됐지만 5분 TTL이 지난 상태로 재개하면 입력 토큰을 전부 다시 캐시에 넣어야 해 비용이 커지기 때문.

## 캐시 친화적 프롬프트 작성법 (예시)

- 나쁜 배치: 시스템 프롬프트 → 현재 시간 → CLAUDE.md → 세컨드 브레인 → 유저 메시지
  - 현재 시간은 매번 바뀌므로, 그 뒤의 CLAUDE.md·세컨드 브레인까지 매번 캐시 미스가 나 수만 토큰을 재계산 → 매우 비효율.
- 좋은 배치: 현재 시간을 뒤로 빼서, 잘 안 바뀌는 시스템 프롬프트·CLAUDE.md·세컨드 브레인을 앞의 고정값으로 묶어 싸게 캐시.
- 클로드 코드는 시스템 프롬프트가 이미 잘 짜여 있지만, 오픈소스 LLM/에이전트는 이런 캐싱을 직접 커스터마이즈해야 하므로 원리를 알아야 한다.

## 6. 토큰 최적화 6기법과 안티패턴

### 6가지 최적화 기법

1. 큰 파일은 한 번 읽고 캐시한다 (클로드 코드가 알아서 해줌)
2. CLAUDE.md를 프롬프트 상단에 둔다 (클로드 코드가 알아서 해줌)
3. 무거운 탐색은 무조건 서브에이전트(subagent)에 위임한다. 탐색은 입력보다 출력 토큰이 커서 소모가 크고, 메인 컨텍스트에 쓸데없는 정보가 쌓이면 메인 세션 컨텍스트가 오염된다. 메인 세션 컨텍스트를 지키는 게 진짜 중요. (요즘 클로드 코드가 알아서 위임하는 추세지만 알아둘 것)
4. 장기 세션은 콤팩트로 적시 압축 — 단, 콤팩트보다 새 세션을 여는 게 훨씬 낫다(강사는 콤팩트를 거의 안 씀)
5. 작업 성격에 맞게 모델을 선택 — 단, 절대 세션 중간에 바꾸지 마라(캐시 소실)
6. 엔진 호출 결과를 태스크별로 재사용하라 — 결과가 같으면 재사용, 결국 캐시와 같은 얘기. 의외로 효과가 큼.

### 안티패턴 (하면 안 되는 것)

- 대용량 파일을 통째로 읽기 — 세션이 한 시간 만에 소진되는 주원인.
- CLAUDE.md/가이드/AI 지도 없이 큰 코드베이스를 그냥 읽게 하기 → 세션의 50%가 금방 참.
- 같은 컨텍스트를 반복 주입 — 이미 캐시된 걸 다시 입력 토큰으로 넣으면 캐시가 무의미.
- 5분 안에 쓸 일 없는 프리픽스를 캐시하기 → 캐시 라이트 비용만 더 씀.
- 막힐 때마다 클리어(clear) 후 새 세션 — 강사는 꼭 나쁘게 보지 않음. 관계없는 작업이면 컨텍스트가 적을수록 클로드 코드가 더 잘 퍼formance. 다만 돈 아끼려다 품질이 떨어져 오히려 돈을 더 쓰는 양날의 검일 수 있음(균형 필요).
- 매 세션마다 CLAUDE.md를 새로 로딩 — 캐시가 전혀 안 됨.
- 설계든 타이핑이든 항상 Opus — Opus가 훨씬 비쌌. 모델 선택이 중요. 서브에이전트는 무조건 Sonnet/Haiku 권장.
- 끝난 작업 노드를 컨텍스트에 계속 남기기 — 필요 없으니 제거.

## 7. 스킬 실습: 토큰 대시보드 (세션 로그 분석)

- 토큰 스킬: 클로드 코드 세션의 JSONL 로그를 파싱해, 리포(repo) 단위로 토큰 사용·캐시 효율·비용·점수를 집계하고 HTML 대시보드와 절감안을 생성한다. 단일 세션이 아니라 여러 세션 전체의 집계·점수화·시각화가 필요할 때 사용.
- 원리: Anthropic CLI가 세션당 usage를 JSONL로 기록해 둔다(입력 토큰, 출력, cache creation input token, cache read 등). 회사는 이 세션 로그를 읽어 최적화해야 한다(“읽는 회사와 안 읽는 회사는 차원이 다르다”).
- 대상 리포는 명시하면 그걸 쓰고, 아니면 현재 작업 디렉토리 기준.

### 채점 스크립트(analyze\_session)의 지표와 루브릭

스크립트가 세션별로 입력 토큰, 캐시 크리에이션/캐시 리드/출력 토큰, cost(USD) 등을 계산해 점수화한다. 핵심 축:

| 지표                       | 의미                       | 판정 기준                                       |
|--------------------------|--------------------------|---------------------------------------------|
| 캐시 유틸라이제이션(캐시 리드/총 입력)   | 캐시 재사용률 (가장 중요, 40% 가중치) | 비율 0.85 이상이면 만점(40점). 높을수록 매 턴 토큰 재전송 비용 감소 |
| 아웃풋 덴시티(output density)  | 총 입력 대비 출력 비율            | 너무 낮으면 읽기만 많고 산출이 없다는 뜻                     |
| 리드 리던던시(read redundancy) | 같은 파일을 여러 번 읽음           | 중복 읽기는 감점 (카토그래피/지도로 상당 부분 해결됨)             |
| 툴 이코노미(tool economy)     | 출력 1K 토큰당 툴 호출 수         | 2~10이 건강한 범위, 20이면 탐색에 토큰을 과소비              |

- 이 루브릭·가중치도 회사/모델별로 조금씩 다를 수 있으며, 자유롭게 수정·추가·삭제해 쓰면 된다(“완성 스킬이 아니다”).

### 데모 대시보드 결과 해석

- “Claude Code 세션 효율 리포트”: 활성 세션 31개, 누적 비용 347불, 세션당 평균 11불, **캐시 적중률 89%(좋은)**, 세션당 평균 도구 호출 13회, 평균 종합 점수 90점.
- 단, 작업을 거의 안 한 프로젝트라 점수가 좋게 나온 것. 작업이 많은 실제 프로젝트는 점수가 점점 내려가고 문제가 드러난다.
- 등급 A+ 세션 2개, B+ 세션 3개. 비용이 큰 세션(약 100불, 54불)은 앞서 SDD(스펙 주도 개발)를 돌린 세션으로 추정.
- 제시된 **비용 절감/개선 액션** 예:
  - 캐시 TTL 5분 단기 세션 즉시 감소(적용)
  - CLAUDE.md에 “200K 토큰 초과 시 콤팩트”(더 빨리 콤팩트) 명시
  - 기본 모델을 Sonnet으로(Opus 과다 사용으로 비쌈)
  - 이미지 업로드 후 즉시 콤팩트(이미지가 토큰을 많이 차지)
  - grep/glob 우선 탐색 규율을 CLAUDE.md에 넣기
- 핵심은 **리포트를 100번 봐도 소용없고 반영해야 의미가** 있다는 것. 자동화해서 적용까지 만드는 것이 이상적(강사가 숙제로 남김).

## 8. 코스트 게이트(cost gate): 사전 예방 + 실시간 모니터링

- 대시보드 vs 코스트 게이트**의 차이:
  - 대시보드 = 이미 쓴 세션 비용을 보여주는 **사후(post-hoc)** 도구(이미 지나간 비용을 개선용으로 분석)
  - 코스트 게이트 = **사전 예방(prevention)**과 **실시간 모니터링** — 예산 초과 직전에 신호를 줌
  - 둘 다 있으면 좋음 (강사의 회사도 둘 다 갖추)
- 세션 단위 게이트**: 한 세션이 일정 토큰을 넘으면 훅(hook)으로 경고/강제. 예: 200K 넘으면 오토 콤팩트하도록 CLAUDE.md에 명시, 또는 SessionStart 등에서 “이번 세션 비용이 초과됐습니다. 계속 진행하시겠습니까?”
- PR 단위 게이트**: 한 PR을 만드는 데 든 누적 세션 비용이 임계를 넘으면 PR에 **라벨을 부착**. 나중에 라벨 붙은 PR들을 모아 분석·최적화 가능.
- 훅으로 누적 토큰을 추적하고 임계점을 넘으면 강제하는 식으로, 실시간·사전 최적화를 둘 다 갖추는 게 좋다. 이런 것들을 팀 플러그인 (푸인)에 추가하라.

## 핵심 요약

- CLAUDE.md가 “지침”이라면 코드베이스는 “지형”이다. 지형이 미로면 좋은 지침도 소용없다.
- 에이전트는 매 세션 신규 입사자다. 실패의 대부분은 코드 품질이 아니라 **탐색·이해 비용**(토큰의 ~80%가 탐색에 낭비)이며, 모델을 키운다고 해결되지 않는다.
- AI 레디 코드베이스는 **세니티(테스트·컨벤션·데드코드/스멜 제거) → 카토그래피(진입점·의존성 그래프·용어집·가치 지도)** 순서로 만든다.
- 테스트는 에이전트의 셀프 검증 신호, 컨벤션 일관성은 패턴 일반화, 데드코드 제거는 오답 경로 차단이라는 새 의미를 갖는다.
- AI Readiness Cartography 스킬로 코드베이스를 100점·7카테고리로 점수화하고, **크론잡으로 주기화 + ROI를 PR로 반영·머지**해야 효과가 있다.
- 비용을 실제로 깎는 최대 도구는 **프롬프트 캐시**다. 캐시 리드는 입력의 약 1/10 가격.
- 캐시는 프리픽스 기반·TTL 5분. **세션 중간에 CLAUDE.md나 모델을 절대 바꾸지 마라**(캐시 전멸). 현재 시간 같은 가변값은 프리픽스 뒤로 빼라.
- 무거운 탐색은 서브에이전트에 위임해 메인 컨텍스트를 지키고, 서브에이전트는 Sonnet/Haiku를 쓴다. 콤팩트보다 새 세션이 낫다.

- 토큰 대시보드 스킬로 세션 JSONL 로그를 분석하며, **캐시 유틸라이제이션(40% 가중치)**이 가장 중요한 채점 축이다.
- 대시보드(사후)와 코스트 게이트(사전 예방·실시간)를 함께 두고, **흑으로** 임계 초과를 강제하라.
- 절약 구호는 소용없다. 이 모든 것을 **팀 레벨 공유 자산(플러그인·흑·스킬)**으로 박아두어 팀·부서·회사 전체 효율을 올리는 것이 목표다.

## 복습 Q&A

**Q. AI 레디 코드베이스와 그냥 “좋은 코드베이스”는 어떻게 다른가?** A. AI 레디 코드베이스는 에이전트가 길을 잃지 않고 작업할 수 있는 코드베이스다. 사람은 익숙해지며 코드 지도를 머릿속에 그리지만 에이전트는 매 세션 처음 들어오므로, 첫 1분 안에 어디로 갈지 명확해야 하는 더 뻑뻑한 기준이 요구된다.

**Q. 에이전트 실패의 주원인은 무엇인가?** A. 코드를 잘못 짜서가 아니라, 올바른 위치·기존 유틸·진짜 진입점을 못 찾는 탐색·이해 비용 문제다. 토큰의 약 80%가 탐색에 낭비되며, 모델이 뚝뚝해진다고 해결되지 않는다.

**Q. AI 레디로 만드는 두 단계와 그 순서가 중요한 이유는?** A. 1단계 세니티(코드 건강성 확보), 2단계 카토그래피(지도 제작)다. 건강하지 않은 코드 위에 지도를 그리면 지도가 거짓말을 하기 때문에 세니티가 반드시 선행돼야 한다.

**Q. 에이전트 시대에 테스트 커버리지의 의미가 어떻게 달라지나?** A. 사람에게는 회귀 방지 안전망이지만, 에이전트에게는 루프를 돌며 스스로 잘못을 알아채는 셀프 검증 신호가 된다. 잘 커버된 코드베이스일수록 에이전트 퍼포먼스가 좋다.

**Q. 프롬프트 캐시란 무엇이고 왜 비용에 결정적인가?** A. 같은 프리픽스를 반복 전송하면 서버가 계산 결과를 캐시해, 다음 호출에서 입력 토큰 대신 캐시 리드(약 1/10 가격)로 읽는다. CLAUDE.md·시스템 프롬프트처럼 반복되는 부분에서 비용을 크게 줄이는 최대 도구다.

**Q. 세션 중간에 CLAUDE.md나 모델을 바꾸면 왜 안 되나?** A. 둘 다 캐시 프리픽스에 포함되므로, 바꾸면 프리픽스가 달라져 앞서 쌓인 캐시가 전부 날아간다. 그러면 처음부터 다시 입력 토큰으로 계산해 큰 손해가 난다. CLAUDE.md는 작업이 끝난 뒤 고쳐야 한다.

**Q. 토큰 대시보드 채점에서 가장 중요한 지표는?** A. 캐시 유틸라이제이션(캐시 리드 / 총 입력)으로 40% 가중치를 가진다. 비율이 0.85 이상이면 만점이며, 재사용률이 높을수록 매 턴 토큰 재전송 비용이 준다.

**Q. 대시보드와 코스트 게이트의 차이는?** A. 대시보드는 이미 쓴 비용을 보여주는 사후 도구이고, 코스트 게이트는 예산 초과 직전에 **흑으로** 경고·강제하는 사전 예방·실시간 도구다. 세션 단위(토큰 임계 경고)와 PR 단위(비용 초과 PR 라벨링)로 적용할 수 있다.

## 용어 정리

- **CLAUDE.md**: 에이전트에게 행동 방식을 알려주는 지침 파일(“헌법”).
- **카토그래피(cartography)**: 에이전트를 위한 지도 제작. 진입점, 의존성 그래프, 도메인 용어집, 가처(함정 패턴)를 담는다.
- **가처(gotcha)**: 잘못 빠지기 쉬운 함정 패턴.
- **세니티(sanity)**: 코드의 건강성(테스트·컨벤션·데드코드·코드 스멜 상태).
- **ADR (Architecture Decision Record)**: 아키텍처 결정 기록. 암묵 지식을 외부화하는 문서.
- **BPE (Byte Pair Encoding)**: 문자열을 토큰으로 나누는 인코딩 방식.
- **프롬프트 캐시 / TTL**: 반복 프리픽스의 계산 결과를 저장하는 캐시. TTL(Time To Live)은 약 5분.
- **캐시 리드 / 캐시 라이트**: 캐시된 프리픽스를 싸게 읽는 것 / 새 정보를 캐시에 넣는(더 비싼) 것.
- **콤팩트(compact)**: 장기 세션의 컨텍스트를 압축하는 기능.
- **서브에이전트(subagent)**: 무거운 탐색 등을 위임받아 메인 컨텍스트를 보호하는 보조 에이전트(Sonnet/Haiku 권장).
- **코스트 게이트(cost gate)**: 비용 임계 초과를 사전에 막는 실시간 통제 메커니즘.
- **SDD (Spec-Driven Development)**: 스펙 주도 개발. 롱러닝 특성상 토큰 효율이 좋은 편.